Zero-Touch Selection of Virtual Network Embedding Algorithms using Multi-Objective Decision Trees

Blind Review[1] [a] and Blind Review[1] [b]
[1] *Blind Review*
{*first_author, second_author*}*@blind.review*

Keywords:
Virtual Network Embedding, Decision Trees, Multi-Objective Optimization, Zero-Touch Administration, Interpretable Machine Learning

Abstract:
The *Virtual Network Embedding* (VNE) problem addresses the NP-hard challenge of mapping virtual network requests onto shared physical infrastructures while optimizing competing objectives. Since no single algorithm performs best in all scenarios, this work proposes multi-objective decision trees to automatically select the most appropriate VNE algorithm for each request based on network state and request characteristics. Three specialized models, each optimized for a specific objective (acceptance rate, revenue-cost ratio, and average revenue), allow adaptation to operational priorities at runtime. Experiments on three *datacenter* topologies with 8,000 requests show top-3 accuracy between 79.1% and 85.9%, with an improvement of +32.9 percentage points in acceptance rate over the fixed *baseline*. The approach provides interpretability through explicit decision paths and inference latency below 1 ms, aligned with zero-touch administration requirements.

1 Introduction

The *Virtual Network Embedding* (VNE) problem has emerged as an essential enabler for modern networks, including cloud datacenters, edge computing, and 5G networks, due to its flexibility and scalability. VNE has become critical in infrastructures where providers share physical network resources among multiple tenants. The problem addresses the challenge of mapping virtual network requests (VNRs) to the physical infrastructure, satisfying capacity constraints, and optimizing objectives such as acceptance rate, processing time, and energy consumption. Over the past two decades, numerous VNE algorithms have been proposed, ranging from exact methods based on Mixed Integer Programming (MIP) to heuristics and meta-heuristics. A fundamental challenge remains: *no single algorithm presents consistently superior performance in all scenarios*. Different VNR characteristics (size, topology, demands) and physical network states (utilization, resource fragmentation) require distinct policies. Traditional VNE deployment approaches typically select a single algorithm for all requests, regardless of their characteristics or the current state of the network. This strategy does not take advantage of the strengths of different algorithms for specific workload scenarios.

Additionally, zero-touch administration of virtualized networks has become a critical requirement in modern infrastructures. The concept of *Zero-Touch Network and Service Management* (ZSM), formalized by the ETSI standard since 2017 (ets, 2017), refers to the ability of systems to operate autonomously through programmable control mechanisms, virtualization, and closed-loop feedback, with minimal human intervention. Industry research indicates that autonomous networks can significantly reduce operational costs. However, the gap between research and deployment remains: while deep Reinforcement Learning (RL) and optimization systems achieve high performance, they lack interpretability and suffer from decision latency inadequate for real-time systems (White et al., 2024). Therefore, systems

[a] https://orcid.org/0000-0000-0000-0000
[b] https://orcid.org/0000-0000-0000-0000

that perform automatic VNE algorithm selection without operator intervention are fundamental to achieving truly autonomous operation.

In many computational domains, the problem of automatically selecting algorithms has been extensively studied (Rice, 1976), showing that different instances benefit from different methods and that portfolio-based selectors often outperform single fixed algorithms (Xu et al., 2012). Surveys on automated algorithm selection (Kotthoff, 2012; Kerschke et al., 2019) and meta-learning (Vanschoren, 2018) report that data-driven selectors and portfolio methods are effective at adapting to heterogeneous instances, but typically rely on complex or black-box models whose internal decision logic is hard to interpret.

Recent knowledge-based schedulers for HPC data centers (Diel et al., 2025) follow a similar philosophy, learning from historical scheduling decisions to combine existing policies and improve performance metrics such as bounded slowdown and waiting time. However, these approaches focus primarily on aggregate performance indicators and rarely provide explicit, human-readable rules that explain *why* a particular algorithm or policy is selected under a given contextual state. In the context of Virtual Network Embedding and Zero-Touch administration, this lack of interpretability is a critical limitation (White et al., 2024), since operators and regulators require auditable decision rules rather than opaque recommendations.

Against this background, this work can be seen as bringing together two lines of research: automatic algorithm selection and interpretable network automation. Similar to portfolio and knowledge-based schedulers, the proposed method learns from historical executions of existing VNE algorithms to build a selector that adapts to request characteristics and network state.

At the same time, by using decision trees the approach obtains explicit decision paths that can be inspected, audited, and translated into natural-language rules. This combination directly addresses the gap in current automatic selection approaches, which improve performance but do not explain *why* decisions are made, and aligns with ZSM requirements for transparent, accountable automation.

In this context, the present work proposes an approach based on multi-objective decision trees to automatically select the most appropriate VNE algorithm for each request, considering the network state and request characteristics. Unlike a single model, the method employs three specialized models, each optimized for a specific objective (acceptance rate, revenue-cost ratio, average revenue), allowing adaptation to operational priorities at runtime, with inference latency below 1 ms enabling online operation.

This work is organized as follows: Section 2 presents the theoretical background; Section 3 discusses related work; Section 4 describes the methodology; Section 5 presents experimental results; and Section 6 presents conclusions.

2 Background and Motivation

2.1 Virtual Network Embedding

Virtual Network Embedding (VNE) is the process of mapping virtual network requests onto a shared physical network infrastructure (Fischer et al., 2013). A virtual network request $G_v = (N_v, L_v)$ consists of a set of virtual nodes N_v and virtual links L_v, where each node and link has specific resource demands (CPU, memory, bandwidth). The physical network $G_p = (N_p, L_p)$ provides the computational and network substrate. The VNE problem can be decomposed into two subproblems: (i) mapping virtual nodes to physical nodes; and (ii) mapping virtual links to paths in the physical network. The objective is to maximize metrics such as VNR acceptance rate, minimize processing time, and optimize physical resource consumption. The VNE problem is NP-hard (Fischer et al., 2013), motivating the development of various algorithm classes:

- Exact Algorithms: Methods such as Mixed Integer Programming (MIP) guarantee optimal solutions, but have exponential complexity.
- Heuristics: Greedy algorithms that make local decisions based on node *rankings*, offering reduced execution time with variable solution quality.
- Meta-heuristics: Techniques such as genetic algorithms (GA), particle swarm optimization (PSO), and Monte Carlo tree search (MCTS) employ stochastic search that balances exploration and quality.

Given that different VNE algorithms present variable performance depending on context, a mechanism becomes necessary to automatically select the most appropriate algorithm. The next

section presents decision trees as a machine learning technique suitable for this contextual selection.

2.2 Decision Trees for Classification

Decision trees are supervised learning models that recursively partition the feature space to create homogeneous prediction regions. The tree starts at a root node containing all training data. Progressively, at each level, the tree subdivides the feature space using binary decisions, creating specialized branches. Each leaf (terminal node) represents a final prediction, in this context, which VNE algorithm is most appropriate.

At each internal node, the tree selects a feature and a threshold, dividing data into two subsets. The splitting criterion uses Gini index or information gain, maximizing class isolation. Each path from root to leaf represents an interpretable decision rule. Figure 1 illustrates an example of a decision path, showing how each intermediate decision in the tree progressively reduces the solution space until reaching a final recommendation. The tree evaluates the current state of both the Physical Network (PN) and the Virtual Network Request (VNR) through a series of decision nodes.

The highlighted path (shown in green) illustrates a concrete example where the tree recommends the MIP algorithm. Starting from the root node containing all training data, the tree first evaluates whether the physical network's node utilization is below or equal to 60%. Following the affirmative branch, the second decision examines whether the VNR size contains 5 nodes or fewer. For small-scale requests in low-utilization networks, a third decision evaluates whether node utilization exceeds 70%, identifying a congestion-prone scenario. This specific combination of conditions leads to the recommendation of MIP (Mixed Integer Programming), an exact algorithm guaranteed to find optimal solutions in congestion scenarios.

Decision trees are chosen in this work not only for their simplicity, but because they directly address the limitations observed in prior automatic algorithm selection approaches. Similar to portfolio-based selectors (Xu et al., 2012) and knowledge-based schedulers for HPC data centers (Diel et al., 2025), the proposed method learns from historical executions of existing algorithms, consolidating algorithmic expertise into a data-driven selector.

Unlike regression-based scoring functions and deep reinforcement learning policies (Cao and Li, 2021; Wang et al., 2024a), however, each prediction produced by a decision tree corresponds to an explicit rule that can be written as a human-readable conditional statement linking measurable network and request characteristics to a specific algorithm recommendation. This property enables per-topology and per-context analysis of the learned selection policy, which is essential for ZSM-compliant operation (White et al., 2024) and regulatory audits.

In addition to interpretability, decision trees provide low inference latency and modest training requirements. In these experiments, prediction latency remains in the sub-millisecond range, making the approach suitable for online VNE decisions without introducing additional bottlenecks. Trees can also be trained with relatively small datasets and limited hyperparameter tuning when compared to deep learning and ensemble methods. This aligns with prior work comparing decision tree learners, Bayesian classifiers, nearest neighbor approaches, and neural networks, where C4.5 decision trees were chosen for their interpretability even when neural networks might achieve higher performance (Cook and Varnell, 1997).

Alternative models such as random forests, gradient-boosted trees, or deep RL may achieve marginally higher predictive accuracy in some scenarios, but they reintroduce black-box behavior and make the extraction of concise decision rules substantially more difficult. Given the emphasis on explainability, auditability, and zero-touch operation in this work, decision trees offer a favorable trade-off between performance, latency, and transparency.

2.3 Multi-Objective Optimization

The selection of which algorithm to use should not consider only a single performance metric. In real production environments, multiple objectives compete simultaneously: VNR acceptance rate, resolution time, and resource consumption. Approaches that optimize a single objective frequently neglect others, causing system imbalance. The proposed strategy employs multiple specialized trees, each optimized for a specific objective, allowing the system to adapt priorities at runtime according to operational requirements.

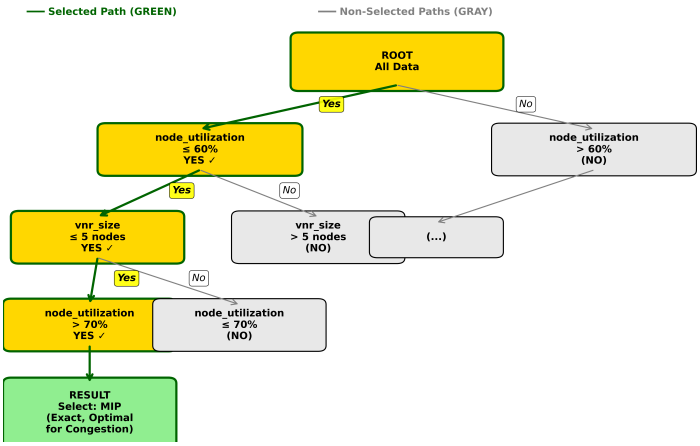

Figure 1: Example of decision path (root to leaf) in tree for VNE algorithm selection.

3 Related Work

3.1 Algorithm Selection and Meta-Learning

The algorithm selection problem was formalized by Rice (Rice, 1976), who established the conceptual *framework*: given a problem space, a set of algorithms, a descriptive *feature* space, and performance metrics, determine which algorithm is most appropriate for each instance. Portfolio-based selectors such as SATzilla (Xu et al., 2012) demonstrate that dynamic selection produces significant gains over fixed algorithms, while surveys on automated algorithm selection (Kotthoff, 2012; Kerschke et al., 2019) and meta-learning (Feurer and Hutter, 2019; Vanschoren, 2018) confirm that data-driven selectors consistently outperform single baselines by adapting decisions to instance-level features. In the HPC context, knowledge-based schedulers (Diel et al., 2025) learn from historical traces to score scheduling policies, improving indicators such as bounded slowdown and waiting time.

Regarding multi-objective optimization, the works (Song et al., 2020) and (Belgacem et al., 2016) explore multi-objective VNE and resource allocation in virtualized networks using evolutionary algorithms or dynamic integer programming, which suffer from computational complexity during *online* operation. The proposed approach differentiates itself by employing independent models for each objective, allowing operators to select at runtime which priority to optimize.

3.2 ML-based Network Automation and Zero-Touch

Deep Reinforcement Learning (DRL) has been applied to network automation: (Cao and Li, 2021) address dynamic resource allocation in NFV, while FlagVNE (Wang et al., 2024a) proposes a flexible RL *framework* for VNE with multiple policies. Although DRL offers adaptability, it faces critical challenges for production: long training time, lack of interpretability, unstable convergence, and high inference latency (White et al., 2024). Similarly, RSCAT (Diel et al., 2023) proposes zero-touch congestion control using actor-critic RL and software-defined networking, showing that combining *machine learning* and classical techniques can enable autonomous operation.

However, existing learning-based solutions, including DRL controllers (Cao and Li, 2021; Wang et al., 2024a) and knowledge-based schedulers (Diel et al., 2025; Diel et al., 2023), predominantly expose aggregate performance indicators without revealing the internal reasoning behind each decision. In the context of Zero-Touch Network and Service Management (ZSM), high predictive accuracy alone is not sufficient: automated controllers must also provide transparent and auditable decisions (White et al., 2024). This gap between opaque decision processes and the need for accountable automation motivates the use of decision trees, which learn from historical data while producing explicit decision paths that operators and regulators can inspect and validate.

4 Methodology

The methodology consists of four main steps:

1. **Data Collection**: Execute eight VNE algorithms implemented in the Virne simulator (Wang et al., 2024b)—representing three algorithm classes: exact methods (*MIP*), meta-heuristics (*GA-Meta, PSO-Meta, SA-Meta, MCTS*), and heuristics (*PL-Rank, RW-Rank-BFS, D-Round*)—on three distinct physical network topologies: two structured *datacenter* topologies (*Tree, Fat-Tree with k=4*) and one random topology. This diversity of algorithms and topologies ensures the decision tree has sufficient variability to learn contextual patterns and generalize across different network scenarios. Detailed performance data are collected in each scenario for model training, validation, and testing.
2. Feature Engineering: Extract 27 relevant characteristics from VNRs, network state, and topological context.
3. Multi-Objective Training: Create three specialized models, Request Acceptance Rate (RAC), Long-term Revenue-to-Cost Ratio (LRC) and Long-term Average Revenue (LAR), each optimizing specific performance criteria.
4. Online Prediction: For each new VNR, use the extracted context to dynamically select the most appropriate VNE algorithm through the model that optimizes the desired criterion.

4.1 Simulation Data Generation

Data were collected using the Virne (Wang et al., 2024b) platform, a comprehensive benchmarking framework for Network Function Virtualization Resource Allocation (NFV-RA) problems. Virne offers highly customizable simulations for various network scenarios, including cloud datacenters, edge computing, and 5G networks. The platform implements more than 30 NFV-RA algorithms in a modular and extensible architecture, providing an *event-driven* simulation environment that models the physical network (PN) infrastructure and sequential virtual network requests (VNRs). Each VNR arrival is treated as a discrete event, creating an instance that the NFV-RA algorithm must solve, evaluating solution feasibility and updating network resources.

Simulations were executed with eight VNE algorithms on three distinct topologies, commonly used in *datacenters*:

- Tree: A binary tree with 32 hosts and 31 switches (63 nodes)
- Fat-Tree: *Datacenter* with k=4 (16 servers, 20 switches, 36 nodes)
- Waxman-16: Random network with 16 nodes and 500 links

Simulations were configured with homogeneous resources (CPU and memory) uniformly distributed on physical nodes. VNRs were generated with variable size (uniform distribution between 2 and 10 virtual nodes), 50% connectivity between virtual nodes, and resource demands following realistic distributions. The system simulates sequential VNR arrivals with arrival rate according to Poisson distribution, where each request has a stochastic lifetime. Virne automatically evaluates the feasibility of each solution proposed by the algorithms, verifying node and link capacity constraints, and calculates standardized performance metrics (RAC, LRC, LAR, AST) for each execution.

For each combination of algorithm and topology, ten repetitions were executed with different random seeds, totaling approximately 8,000 unique VNRs after deduplication. This approach ensures statistical robustness and captures the variability inherent to stochastic algorithms and dynamic characteristics of virtual network requests.

4.2 Feature Engineering

A set of 27 features was carefully selected, divided into categories:

- VNR Characteristics (9 features): number of nodes, number of links, size ratio, average demand per node, average demand per link, connectivity, total demand, node/link ratio, expected lifetime.
- Physical Network State (4 features): available resources, node utilization, link utilization, overall utilization.
- System State (3 features): number of active VNRs, system load, active physical nodes.
- Heterogeneity and Fragmentation (10 engineered features): Gini indices for resource distribution, memory fragmentation, topological diversity indices.
- Context (1 feature): topology type.

4.3 Multi-Objective Architecture

Instead of a single classification model, we employ three specialized models, each optimized for a specific objective:

- RAC: Maximize virtual network request acceptance rate
- LRC: Maximize long-term revenue-cost ratio
- LAR: Maximize long-term average revenue

Each model is a decision tree trained independently on historical simulation data. During online operation, the operator (or automatic policy) selects which objective to prioritize, and the corresponding model predicts the best algorithm.

4.4 Model Training

The training process was performed independently for each objective (RAC, LRC, LAR), allowing each model to specialize in optimizing its specific metric. Data division was stratified by algorithm class to ensure balanced distribution between training (70%), validation (15%), and test (15%) sets. This stratification is crucial to maintain representativeness of all classes in each partition, especially considering the severe imbalance observed in the data.

Models use `DecisionTreeClassifier` from scikit-learn with the following optimized hyperparameters:

- Maximum depth: 10 (balancing expressiveness and interpretability)
- Minimum samples per leaf: 3 (reduced to allow capture of minority classes)
- Minimum samples for split: 6 (allows splits with fewer samples)
- Gini impurity for *feature* selection, measuring class separation with values from 0 for pure nodes to 0.5 for maximum impurity, minimizing impurity to maximize class homogeneity at each split

As shown in Table 1, no algorithm is universally optimal, evidencing the need for contextual selection. This observation justifies the dynamic algorithm selection approach proposed in this work.

Table 1: VNE Algorithm Performance Summary

Algorithm	Acc. Rate	Revenue	R2C	Time (s)
MIP	36.8%	186.8	115.6	1.026
PL_RANK	29.5%	388.4	127.4	0.661
GA_META	27.9%	400.6	139.8	0.603
RW_RANK_BFS	25.8%	373.9	120.4	0.558
SA_META	24.6%	372.6	133.5	0.534
MCTS	23.9%	379.7	167.0	0.468
D_ROUND	20.4%	365.4	246.0	0.357
PSO_META	20.1%	352.1	120.0	0.309

4.5 Multi-Metric Comparison of VNE Algorithms

Different operational contexts (high load, limited resources, latency requirements) require different trade-offs. Figure 2 presents a comprehensive analysis of four complementary metrics for the eight evaluated algorithms:

4.5.1 Class Balancing Techniques

To reduce dataset imbalance, a class balancing techniques was applied. Instead of scikit-learn's default balancing, weights calculated as $w_i = 1/\sqrt{f_i}$ were used, where f_i is the relative frequency of class i. This strategy assigns significantly larger weights to rare classes:

- Rare classes (`pso_meta`, 17 samples): weight 10.0×
- Intermediate classes (`rw_rank_bfs`, 39 samples): weight 6.6×
- Majority classes (`ga_meta`, 987 samples): weight 1.3×

For evaluation, weighted F1-Score and F1-Macro (arithmetic mean of F1 of all classes, treating them equally) metrics were used to evaluate balanced performance between classes.

5 Experimental Results

5.1 Classification Performance

Table 2 presents the classification metrics obtained for the three considered objectives (RAC, LRC, and LAR):

Objective	Accuracy	F1-Score	Top-2	Top-3
RAC (Acceptance Rate)	67.26%	0.68	82.66%	85.90%
LRC (Revenue-Cost Ratio)	54.46%	0.55	74.55%	83.95%
LAR (Average Revenue)	48.14%	0.48	67.59%	79.09%

Table 2: Classification Metrics by Objective (Balanced Models)

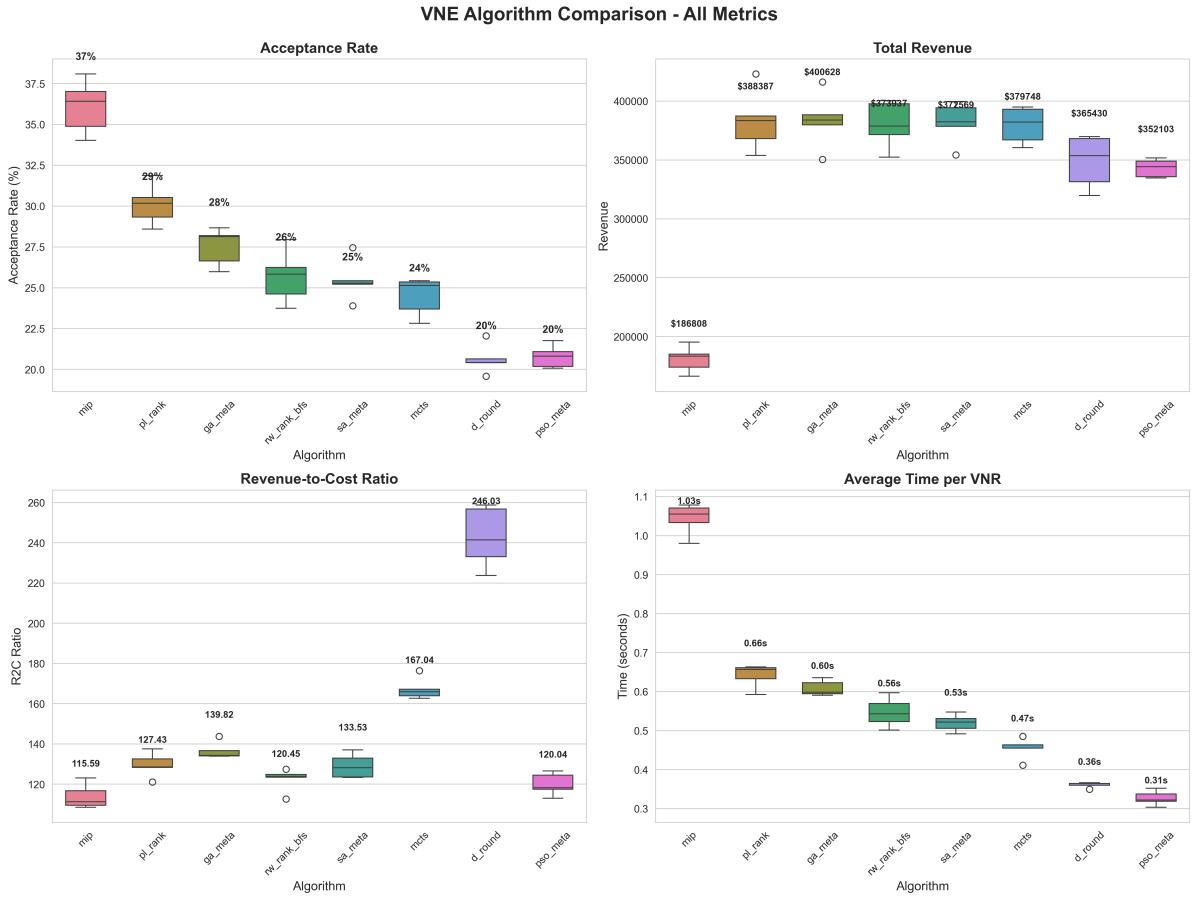

Figure 2: Multi-Metric Comparison: Acceptance Rate, Total Revenue, Revenue-Cost Ratio, and Average Time per VNR. The graphs show distribution of results in five simulated executions per algorithm across topologies (Tree, Fat-Tree, Waxman-16). No single algorithm is optimal in all dimensions.

Results presented in Table 2 indicate that the RAC objective presents better performance in all metrics, with accuracy of 67.26% and top-3 accuracy of 85.90%. The LRC objective presents intermediate performance, while LAR presents lower accuracy (48.14%), but still with top-3 accuracy of 79.09%, indicating that the model frequently identifies the best algorithm among the top three options. The performance difference between objectives reflects the distinct complexity of each metric and the inherent variability of algorithms in different contexts.

5.2 Comparison with *Baseline*

Classification accuracy was measured for each objective, comparing decision tree performance with the fixed algorithm *baseline*. Figure 3 presents the results. It is possible to observe that accuracy depends significantly on the chosen fixed algorithm, evidencing the need for contextual selection.

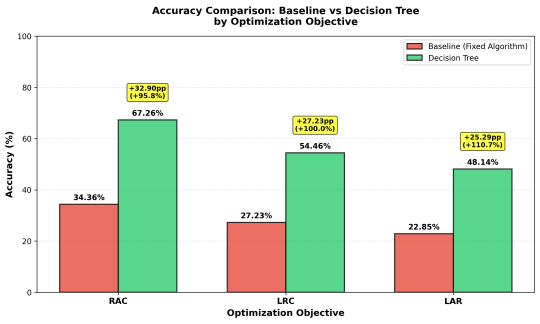

Figure 3: Accuracy Comparison: Baseline vs Decision Tree by Optimization Objective

Figure 3 reveals that contextual selection via decision trees significantly outperforms the use of fixed algorithms. For the RAC objective, the improvement is +32.90 percentage points (+95.8%

relative), while for LRC and LAR the improvements are even more expressive in relative terms (+100.0% and +110.7%, respectively). These results confirm that adaptive algorithm selection based on contextual characteristics is fundamental to optimizing system performance.

To validate that gains in classification accuracy translate into real operational performance improvement, performance was evaluated when decision trees are used for algorithm selection in simulated executions. Figure 4 presents the comparison of real performance (using individual values per VNR) between tree-based selection and fixed algorithm baseline.

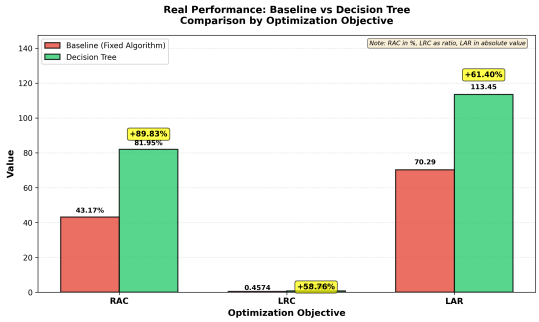

Figure 4: Real Performance: Tree vs Baseline by Optimization Objective

Results show that contextual selection via decision trees produces substantial improvements in real operational performance in all objectives:

- RAC: Acceptance rate increases from 43.17% (fixed algorithm) to 81.95% (contextual selection), representing an improvement of +89.83%.
- LRC: Revenue-cost ratio increases from 0.4574 to 0.7262, with an improvement of +58.76%.
- LAR: Average revenue increases from 70.29 to 113.45 per VNR, with an improvement of +61.40%.

Results show that contextual selection via decision trees is significantly superior to using a fixed algorithm, with substantial improvements in all evaluated objectives. The approach captures important patterns of algorithm selection based on contextual characteristics of VNRs, allowing dynamic adaptation to different network conditions and resource requirements.

5.3 Interpretability Analysis

One of the main advantages of the approach is that each decision can be traced through the decision tree.

Unlike deep neural networks (black box), operators can understand **exactly why** an algorithm was selected:

"If VNR size < 30% of physical network AND node utilization > 70%, then select MIP (exact, accepts in congestion). Otherwise, if connectivity > 0.5, select GA-Meta."

Additionally, *feature* importance analysis reveals which characteristics are most relevant for each objective. For RAC (Request Acceptance Rate) and LRC (Long-Term Revenue-to-Cost), resource efficiency ($resource\_efficiency$) is the most critical *feature*, with importance of 0.13 and 0.11 respectively. In the case of LAR (Long-Term Average Revenue), VNR link demand ($v\_net\_demand\_per\_link$) and total demand ($v\_net\_total\_demand$) are the dominant *features*. These findings confirm that different objectives prioritize distinct characteristics of VNRs and physical network state. Figure 5 exemplifies *feature* importance for the RAC objective, while Figures 6 and 7 present results for the remaining objectives.

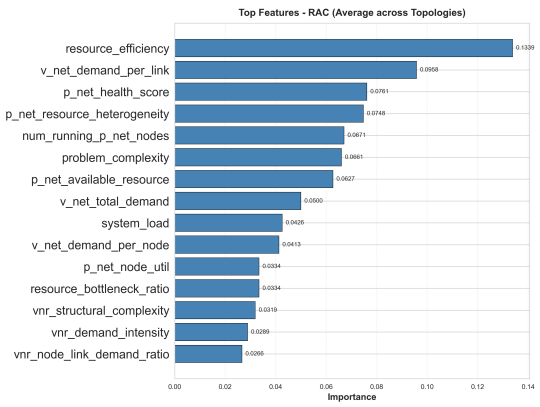

Figure 5: Feature Importance - RAC (Request Acceptance Rate). The top 15 most important features (average across topologies) for the RAC objective. Resource efficiency ($resource\_efficiency$) is the most critical feature with importance of 0.134, followed by VNR link demand ($v\_net\_demand\_per\_link$) with 0.096.

This interpretability is critical, as it allows regulators to perform audits by verifying decisions made by the system, facilitates debugging work by enabling operators to understand system behavior, and increases confidence and acceptance of zero-touch automation.

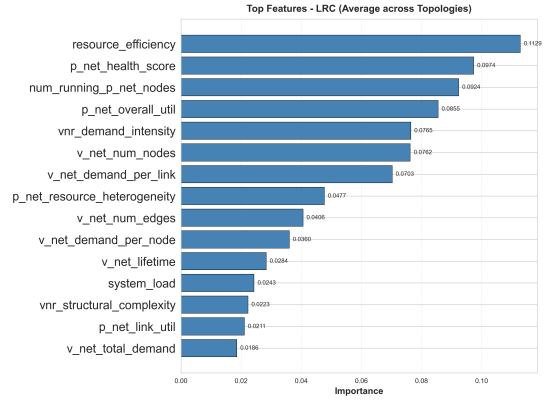

Figure 6: Feature Importance - LRC (Long-Term Revenue-to-Cost). The top 15 most important features (average across topologies) for the LRC objective. Resource efficiency ($resource\_efficiency$) is the most critical feature with importance of 0.113, followed by physical network health score ($p\_net\_health\_score$) with 0.097.

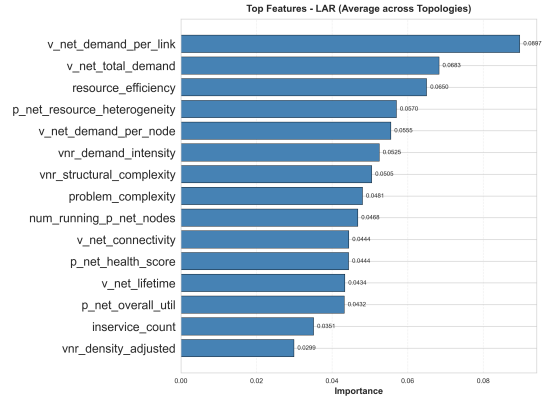

Figure 7: Feature Importance - LAR (Long-Term Average Revenue). The top 15 most important features (average across topologies) for the LAR objective. VNR link demand ($v\_net\_demand\_per\_link$) is the most important feature with 0.090, followed by total demand ($v\_net\_total\_demand$) with 0.068.

5.4 Inference Latency

Prediction latency was measured over 8,000 iterations, yielding an average of 0.034 ms per prediction, with p99 at 0.055 ms and worst case at 0.119 ms. More than 99% of predictions complete in less than 0.06 ms, confirming consistent submillisecond latency. By comparison, FlagVNE requires 50–200 ms per decision (Wang et al., 2024a), representing a difference of three orders of magnitude. This low latency enables online real-time decision making, critical for zero-touch operation.

5.5 Key Observations

The proposed approach achieves acceptance rate improvements of +89.83% (from 43.17% to 81.95%) compared to fixed algorithm baselines, with top-3 accuracy between 79.1% and 85.9%. Revenue-cost ratio and average revenue improved by +58.76% and +61.40%, respectively. The class balancing techniques (custom weights $1/\sqrt{f}$ and hyperparameter tuning) resulted in an F1-Macro of 0.56 for the RAC objective, confirming that contextual selection based on request characteristics and network state is essential since no VNE algorithm is universally optimal.

6 Conclusion

The Virtual Network Embedding (VNE) problem is fundamental in modern cloud infrastructures, where mapping virtual network requests onto physical networks while optimizing multiple objectives is NP-hard. A fundamental challenge in this domain is that no single algorithm presents consistently superior performance in all scenarios, and different VNR characteristics and physical network states require distinct algorithmic approaches. Additionally, modern infrastructures demand zero-touch administration capabilities, requiring systems that can operate autonomously with interpretability and low latency for real-time decision-making.

To address this challenge, this work proposed an approach for zero-touch selection of Virtual Network Embedding algorithms using multi-objective decision trees, considering virtual network request characteristics and physical infrastructure state. Experimental results show that the foundations of the proposed approach (decision trees, multi-objective optimization, and interpretability) are potential candidates to achieve zero-touch network control according to the ETSI ZSM standard. The experimental analysis, based on three datacenter topologies (Tree, Fat-Tree, and Waxman-16) with 8,000 virtual network requests, showed that contextual algorithm selection can reduce processing time and improve acceptance rate in various scenarios, without requiring software updates at infrastructure endpoints.

As limitations, the model was trained on three simulated topologies, and the characteristics capture instantaneous network state. As future work, promising directions include transfer learning between topologies, incorporation of temporal his-

tory of requests, and validation in production environments with real *datacenter* workloads.

ACKNOWLEDGEMENTS

Blind review.

REFERENCES

(2017). Etsi zsm - zero-touch network and service management.

Belgacem, A. et al. (2016). A multi-objective approach for vne problems using multiple ilp formulations. *CLEI Electronic Journal*, 19(2):1–15.

Cao, Z. and Li, Y. (2021). Deep reinforcement learning for resource allocation in network slicing. *IEEE Transactions on Network and Service Management*, 18(1):873–886.

Cook, D. J. and Varnell, R. C. (1997). Maximizing the benefits of parallel search using machine learning. In *Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI-97)*, pages 559–564. AAAI Press.

Diel, G., Miers, C. C., Pillon, M. A., and Koslovski, G. P. (2023). Rscat: Towards zero touch congestion control based on actor–critic reinforcement learning and software-defined networking. *Journal of Network and Computer Applications*, 215:103639.

Diel, G., Nascimento Kraus, A., and Piêgas Koslovski, G. (2025). Knowledge-based job scheduling for hpc. *Cluster Computing*, 28:153.

Feurer, M. and Hutter, F. (2019). Automated machine learning: Methods, systems, challenges. *The Springer Series on Challenges in Machine Learning*, pages 3–33.

Fischer, A., Botero, J. F., Till Beck, M., Figuerola, S., and Sahasrabuddhe, V. (2013). Virtual network embedding: A survey. *IEEE communications surveys & tutorials*, 15(4):1888–1906.

Kerschke, P., Hoos, H. H., Neumann, F., and Trautmann, H. (2019). Automated algorithm selection: Survey and perspectives. *Evolutionary Computation*, 27(1):3–45.

Kotthoff, L. (2012). Algorithm selection for combinatorial search problems: A survey. *arXiv preprint arXiv:1210.7959*.

Rice, J. R. (1976). The algorithm selection problem. *Advances in Computers*, 15:65–118.

Song, A. et al. (2020). A constructive particle swarm optimizer for virtual network embedding. *IEEE Transactions on Network and Service Management*.

Vanschoren, J. (2018). Meta-learning: A survey. *arXiv preprint arXiv:1810.03548*.

Wang, H., Zhang, C., and Wang, R. (2024a). Flagvne: Flexible algorithm grouping for virtual network embedding using reinforcement learning. *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*.

Wang, T., Deng, L., Chen, X., Wang, J., He, H., Ding, L., Wu, W., Fan, Q., and Xiong, H. (2024b). Virne: A comprehensive benchmark for deep rl-based network resource allocation in nfv. *arXiv preprint arXiv:2507.19234*. Available at `https://github.com/GeminiLight/virne`.

White, R., Chen, M., and Kumar, R. (2024). Zero-touch networks: Towards next-generation network automation. *Computer Networks*, 218:110269.

Xu, L., Hutter, F., Hoos, H. H., and Leite, R. (2012). Satzilla: Portfolio-based algorithm selection for sat. In *Journal of Artificial Intelligence Research*, volume 32, pages 565–606.